Test Rapport Titre

Ayath ABOGOUNRIN, Guillaume POTEL, Mathéo VIGNERES

Année Universitaire 2025-2026

Table des matières

Introduction **1**

Introduction **1**

I Généralités **2**

1 Logique des propositions **2**

2 Logique du premier ordre **3**

3 Preuve automatisée **5**

4 Unification **6**

II Structures de données et algorithmes **9**

1 Structures de données représentant un terme **9**
1.1 Problématique 9
1.2 Intuition 9
1.3 Implémentation 9

2 Structures de données représentant un ensemble de termes **10**
2.1 Liste 11
2.2 Ensemble 11
2.3 Dictionnaire 12

3 Algorithmes d'unification **13**
3.1 Robinson 13
3.2 Martelli-Montanari 16

4 Discrimination Tree **17**

III Résultats **22**

1 Protocole de benchmarks **22**

2 Résultats **22**

Introduction

Blablabla

I. Généralités

D'après nos connaissances, l'UE de L3 intitulée Logique des propositions - Logique du premier ordre*, de Jean Lieber et l'UE de M1 intitulée* Logiques et Modèles de Calcul*, de Didier Galmiche.*

Fil conducteur.

Tout au long de ce chapitre, nous allons construire progressivement les outils nécessaires pour répondre à une question concrète : comment une machine peut-elle *démontrer* automatiquement que « Socrate est mortel » d'après les hypothèses « Tout homme est mortel. » et « Socrate est un homme » ? Ce raisonnement, connu sous le nom de *syllogisme*, peut sembler trivial au lecteur mais requiert, pour être mécanisé, des outils formels précis. Nous introduirons d'abord brièvement la logique des propositions, sur laquelle s'appuie la logique du premier ordre, avant de présenter cette dernière puis l'unification, les deux notions centrales de ce rapport.

1. Logique des propositions

La *logique des propositions* (ou logique propositionnelle) repose sur une idée simple : on représente des *faits élémentaires*, par exemple « Socrate est un homme », auxquels on attribue une *valeur de vérité* ($vrai$ ou $faux$). Ces *faits élémentaires* sont appelés *variables propositionnelles*, notées conventionnellement p, q, r, ...Ces *variables* peuvent ensuite être combinés à l'aide de *connecteurs logiques* pour former des *formules*.

Voici une définition formelle d'une *formule* en *logique propositionnelle* :

$$\langle \text{formule} \rangle ::= \langle \text{variable propositionnelle} \rangle \mid \neg(\langle \text{formule} \rangle) \mid (\langle \text{formule} \rangle \ \langle \text{connecteur binaire} \rangle \ \langle \text{formule} \rangle)$$

$$\langle \text{connecteur binaire} \rangle ::= \wedge \mid \vee \mid \rightarrow \mid \leftrightarrow$$

Remarque. *Les connecteurs ont les lectures suivantes :*

$\neg$	$\wedge$	$\vee$	$\rightarrow$	$\leftrightarrow$
« non »	*« et »*	*« ou »*	*« implique »*	*« équivaut à »*

Exemple : traduction du syllogisme.

On peut encoder notre syllogisme en posant :

p	« Socrate est un homme »
q	« Tout homme est mortel »
r	« Socrate est mortel »

et former la *formule* $(p \wedge q) \rightarrow r$, qui se lit « si Socrate est un homme et tout homme est mortel, alors Socrate est mortel ».

Limites.

La logique propositionnelle ne traite que d'un individu à la fois : elle est insuffisante pour exprimer des propriétés générales portant sur des ensembles d'objets ou des relations entre eux.

Revenons à notre syllogisme, on *encode en dur* chaque fait : la proposition q est un bloc opaque ; on ne peut pas en extraire la règle générale « pour tout individu x, si x est un homme, alors x est mortel ». Si l'on voulait raisonner sur d'autres individus, e.g. Platon ou Aristote, il faudrait introduire autant de nouvelles *variables propositionnelles*, sans jamais capturer la généralité du raisonnement.

C'est précisément cette limite que lève la *logique du premier ordre*, présentée dans la section suivante, en introduisant des *variables* sur lesquelles on peut quantifier.

2. Logique du premier ordre

Pour pouvoir étendre le champs de couverture de la *logique propositionnelle*, on a défini la *logique du premier ordre* (ou logique des prédicats).

Cette logique permet la représentation de propriétés portant sur des ensembles d'objets ou des relations. Pour les décrire au sein de *formules*, on se donne un *alphabet* composé des éléments suivants :

— Ensemble de *variables* : $x, y, z, \ldots$ Représentant des individus *indéterminés* ;

On définit également ce que l'on appelle une *signature*, qui rend notre représentation unique, composée de deux ensembles :

— Ensemble de symboles de *fonctions* : $f, g, h, \ldots$ Permettant de construire de nouveaux objets à partir d'objets existants, e.g. $pere(socrate)$ désigne le père de Socrate ;

— Ensemble de symboles de *prédicats* : $P, Q, R, \ldots$ Exprimant des propriétés ou des relations entre objets, e.g. $Homme(socrate)$ exprime que Socrate est un homme.

On associe à ces fonctions et prédicats une *arité* (i.e. le nombre d'arguments qu'ils prennent). Par exemple, *pere* est d'arité 1 car il prend un seul argument, tandis qu'un prédicat comme $EstParentDe(x, y)$ est d'arité 2 car il met en relation deux individus. Parmi les symboles de fonctions, ceux d'arité 0 sont appelés *constantes* :

— Ensemble de symboles de constantes : $a, b, c, \ldots$ Représentant des individus précis et fixes, e.g. *socrate*.

> **Remarque.** *Le choix de ces différents symboles est purement arbitraire : on pourrait nommer les variables a, b ou c. Dans un souci d'harmonisation et de standardisation, on utilisera néanmoins les symboles les plus courants dans la littérature.*

Pour définir la structure d'une *formule* en *logique du premier ordre*, il nous faut définir la notion d'*atome* qui s'appuie elle même sur la notion de *terme*.

Terme.

Un *terme* est une expression désignant un *objet*. Il peut être une *variable*, une *constante* ou un *symbole de fonction* appliqué à d'autres *termes*.

Formellement, on défini un *terme* par la grammaire suivante :

$$\langle \text{terme} \rangle ::= \langle \text{variable} \rangle \mid \langle \text{constante} \rangle \mid \langle \text{symbole de fonction} \rangle \underbrace{(\langle \text{terme} \rangle, \ldots, \langle \text{terme} \rangle)}_{n \text{ termes}}$$

où n correspond à l'arité du symbole de fonction.

Par exemple, avec *socrate* comme constante, x comme variable et *pere* comme symbole de fonction d'arité 1, on peut construire les *termes* :

$$socrate \qquad x \qquad pere(socrate) \qquad pere(x)$$

> **Remarque.** *Un terme* n'est pas *une formule, il désigne un objet, il n'exprime pas une propriété.*

Atome.

Là où un *terme* désigne un *objet*, un *atome* exprime une *propriété* sur des objets. Il est formé d'un *symbole de prédicat* appliqué à des termes. Formellement :

$$\langle \text{atome} \rangle ::= \langle \text{prédicat} \rangle (\underbrace{\langle \text{terme} \rangle, \ldots, \langle \text{terme} \rangle}_{n \text{ termes}})$$

où n correspond à l'arité du prédicat.

Par exemple, avec *Homme* et *Mortel* comme prédicats d'arité 1, *socrate* comme constante et x comme variable :

$$Homme(socrate) \qquad Mortel(socrate) \qquad Mortel(x)$$

sont des atomes. $Homme(socrate)$ exprime que Socrate est un homme, $Mortel(x)$ exprime que l'individu désigné par x est mortel.

> **Remarque.** *Un littéral (ou des littéraux) est un prédicat auquel on a ajouté un signe.*
> *Exemple : $P(f(x))$ et $\neg P(f(x))$*

Formule.

Les *atomes* peuvent être combinés à l'aide des connecteurs logiques vus en *logique propositionnelle*, et enrichis par deux *quantificateurs* :

- le *quantificateur universel* $\forall$ (« pour tout ») : $\forall x\, \varphi$ signifie que φ est vraie quelle que soit la valeur attribuée à x ;
- le *quantificateur existentiel* $\exists$ (« il existe ») : $\exists x\, \varphi$ signifie qu'il existe au moins une valeur de x pour laquelle φ est vraie.

Une *formule* de la *logique du premier ordre* est alors définie par :

$$\begin{aligned}\langle \text{formule} \rangle ::= {} & \langle \text{atome} \rangle \mid \neg(\langle \text{formule} \rangle) \mid \\ & (\langle \text{formule} \rangle\ \langle \text{connecteur binaire} \rangle\ \langle \text{formule} \rangle) \mid \langle \text{quantificateur} \rangle\ \langle \text{variable} \rangle\ (\langle \text{formule} \rangle)\end{aligned}$$

Par exemple, « pour tout individu x, si x est un homme alors x est mortel » s'écrit :

$$\forall x \left(Homme(x) \rightarrow Mortel(x) \right)$$

Ce que la *logique propositionnelle* ne pouvait pas exprimer, i.e. la généralité du raisonnement sur tout individu x, est ici capturé par une unique *formule quantifiée*.

3. Preuve automatisée

Nous disposons maintenant des outils pour formaliser proprement notre syllogisme. Introduisons la *signature* suivante :

— Constante : *socrate*

— Symboles de prédicats : *Homme* d'arité 1, *Mortel* d'arité 1

Les deux prémisses s'écrivent alors :

$\forall x \, \big(Homme(x) \to Mortel(x)\big)$	tout homme est mortel
$Homme(socrate)$	Socrate est un homme

Et l'on cherche à démontrer d'après les deux prémisses :

$Mortel(socrate)$	Socrate est mortel

Remarque. *En logique propositionnelle, il aurait fallu créer une proposition distincte pour chaque individu. Ici, une seule formule quantifiée $\forall x$ capture la règle générale, applicable à n'importe quelle constante.*

Forme clausale et réfutation.

La mise sous *forme clausale* consiste à transformer les formules de la base de connaissances en un ensemble de *clauses*, i.e. des disjonctions de littéraux, qui leur est logiquement équivalent. Cette transformation se fait en plusieurs étapes :

1. éliminer les implications et équivalences ;
2. faire descendre les négations au plus près des prédicats ;
3. standardiser les variables (renommer les variables si nécessaire) ;
4. mettre la formule sous forme prénexe (tous les quantificateurs en tête) ;
5. éliminer les quantificateurs existentiels (via un procédé nommé *skolémisation*) ;
6. supprimer les quantificateurs universels, qui deviennent implicites ;
7. mettre la partie restante sous forme de clauses.

Pour notre syllogisme, la formule $\forall x \, (Homme(x) \to Mortel(x))$ est d'abord réécrite sous la forme $\forall x \, (\neg Homme(x) \vee Mortel(x))$, en utilisant l'équivalence logique entre $A \to B$ et $\neg A \vee B$. Aucune autre transformation n'est ici nécessaire, car la formule est déjà en forme prénexe et ne contient pas de quantificateur existentiel. Les quantificateurs universels étant implicites, on obtient ainsi la clause $\neg Homme(x) \vee Mortel(x)$.

La technique de *réfutation* consiste, plutôt que de démontrer directement une conclusion, à ajouter sa *négation* aux prémisses et à chercher une *contradiction*. Si une contradiction est trouvée, cela prouve que la conclusion est bien vraie.

En ajoutant la négation de $Mortel(socrate)$ à nos prémisses et en les mettant sous forme clausale, on obtient trois clauses :

$$
\begin{aligned}
C_1 &= \neg Homme(x) \vee Mortel(x) \\
C_2 &= Homme(socrate) \\
C_3 &= \neg Mortel(socrate)
\end{aligned}
$$

Il nous reste à dériver une contradiction à partir de ces trois clauses, ce que réalise la *règle de résolution*, présentée ci-après.

> **Remarque.** *Il existe un théorème, que l'on utilise ici, qui nous dit que toute formule de la logique du premier ordre peut être mise sous forme clausale. Cette transformation, bien que non triviale dans le cas général, est toujours possible et constitue une étape préalable indispensable à l'application de la règle de résolution.*

Règle de résolution.

La *règle de résolution* permet de dériver une nouvelle clause à partir de deux clauses existantes. Si une clause contient un littéral L et une autre contient sa négation $\neg L$, on peut en dériver une nouvelle clause, appelée *résolvante*, contenant tous les autres littéraux des deux clauses :

$$\frac{L \vee C \qquad \neg L \vee D}{C \vee D}$$

Cependant, les littéraux ne sont pas toujours syntaxiquement identiques : dans notre exemple, C_1 contient $Homme(x)$ et C_2 contient $Homme(socrate)$. Ces deux littéraux partagent le même symbole de prédicat, mais leurs arguments diffèrent : l'un est une variable x, l'autre une constante *socrate*. Pour pouvoir appliquer la règle de résolution, il faut d'abord les rendre identiques en *substituant* x par *socrate*. C'est précisément ce que réalise l'*unification*, que nous définirons dans la section suivante.

4. Unification

Substitution.

Une *substitution* σ est une fonction qui associe à certaines variables des termes. Par exemple, $\sigma = \{X \mapsto b,\ Y \mapsto f(a)\}$ associe la variable X au terme b et la variable Y au terme $f(a)$. Appliquer σ à un terme t, noté $\sigma(t)$, consiste à remplacer simultanément chaque occurrence des variables concernées par leur terme associé.

Par exemple, si $t = g(X, Y)$, alors :

$$\sigma(t) = \sigma(g(X, Y)) = g(b, f(a))$$

Unification.

L'*unification* consiste à déterminer s'il existe une substitution σ, appelée *unificateur*, telle que $\sigma(t_1) = \sigma(t_2)$ pour deux termes t_1 et t_2 donnés. Si une telle substitution existe, on dit que t_1 et t_2 sont *unifiables*.

Parmi tous les unificateurs possibles, on cherche le plus général, noté *MGU* (de l'anglais *Most General Unifier*), c'est-à-dire celui qui impose le moins de contraintes sur les variables. Il est dit « le plus général » car tout autre unificateur peut s'obtenir en composant le MGU avec une substitution supplémentaire.

Remarque. *L'unification n'est pas toujours possible. Par exemple, les termes $f(a)$ et $g(a)$ ne sont pas unifiables car leurs symboles de tête sont différents. De même, une variable x ne peut pas être unifiée avec un terme qui la contient déjà, comme $f(x)$, car cela produirait un terme de taille infinie.*

Exemple.

Considérons les deux termes suivants :

$$t_1 = f(X, Y) \qquad \text{et} \qquad t_2 = f(a, Z)$$

On cherche une substitution σ telle que $\sigma(t_1) = \sigma(t_2)$. La substitution

$$\sigma = \{X \mapsto a,\ Y \mapsto Z\}$$

unifie ces deux termes :

$$\sigma(t_1) = f(a, Z) = \sigma(t_2)$$

Cette substitution est le MGU, car elle réalise uniquement les substitutions strictement nécessaires. D'autres substitutions permettent également l'unification, par exemple :

$$\sigma_1 = \{X \mapsto a,\ Y \mapsto b,\ Z \mapsto b\}$$

ce qui donne :

$$\sigma_1(t_1) = f(a, b) = \sigma_1(t_2)$$

Ou encore :

$$\sigma_2 = \{X \mapsto a,\ Y \mapsto g(c),\ Z \mapsto g(c)\}$$

pour laquelle :

$$\sigma_2(t_1) = f(a, g(c)) = \sigma_2(t_2)$$

Ces substitutions sont plus spécifiques que le MGU, car elles s'obtiennent en composant σ avec une substitution supplémentaire appliquée à Z. En revanche, la substitution

$$\sigma_3 = \{X \mapsto b,\ Y \mapsto Z\}$$

n'est pas un unificateur, car :

$$\sigma_3(t_1) = f(b, Z) \neq f(a, Z) = \sigma_3(t_2)$$

Ainsi, toute substitution unificatrice s'obtient à partir du MGU

$$\{X \mapsto a,\ Y \mapsto Z\}$$

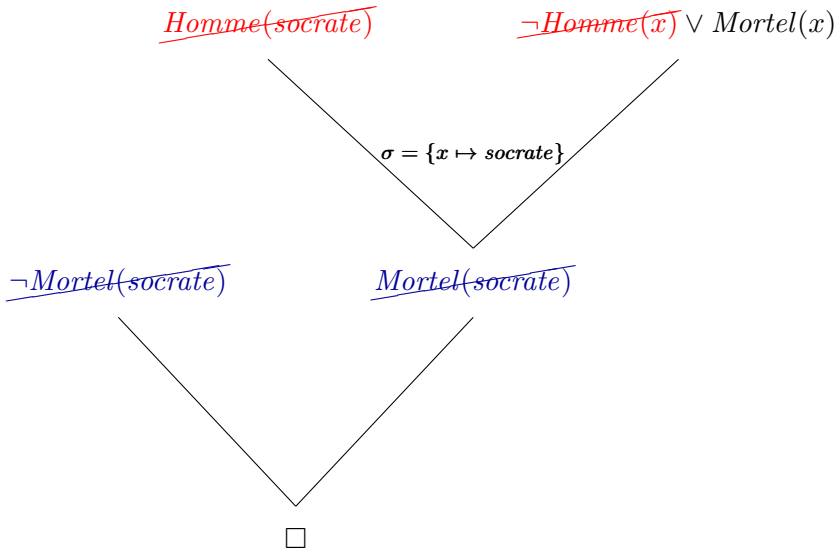
par spécialisation.

Clôture du fil conducteur.

Revenons à nos trois clauses obtenues à la section précédente :

$$\begin{aligned} C_1 &= \neg Homme(x) \vee Mortel(x) \\ C_2 &= Homme(socrate) \\ C_3 &= \neg Mortel(socrate) \end{aligned}$$

Pour appliquer la règle de résolution sur C_1 et C_2, il faut unifier $Homme(x)$ et $Homme(socrate)$. La substitution $\sigma = \{x \mapsto socrate\}$ est leur MGU. On peut alors dériver la contradiction en deux étapes :



■ littéraux résolus à l'étape 1 : $\sigma_1 = \{x \mapsto socrate\}$
■ littéraux résolus à l'étape 2 : $\sigma_2 = \emptyset$

— **Étape 1 :** On résout C_1 et C_2. Les littéraux $Homme(x)$ et $Homme(socrate)$ sont unifiés par $\sigma = \{x \mapsto socrate\}$, ce qui produit la résolvante $Mortel(socrate)$.

— **Étape 2 :** On résout $Mortel(socrate)$ et $C_3 = \neg Mortel(socrate)$. Les deux littéraux sont identiques, aucune substitution n'est nécessaire. On obtient la *clause vide* $\square$, témoin de la contradiction.

La négation de $Mortel(socrate)$ est donc incompatible avec nos prémisses, ce qui prouve que $Mortel(socrate)$ est bien une conséquence du syllogisme. Cet exemple illustre le rôle central de l'unification dans la démonstration automatique : sans elle, la règle de résolution ne pourrait pas instancier les variables et serait donc inutilisable en pratique. La question de l'*efficacité* de cette unification, en fonction des structures de données et des algorithmes utilisés, est précisément l'objet du reste de ce rapport.

II. Structures de données et algorithmes

1. Structures de données représentant un terme

1.1. Problématique

La définition d'un terme en logique du premier ordre impose des contraintes naturelles sur sa représentation : un terme est soit une variable, soit une constante, soit une fonction appliquée à d'autres termes. Toute structure de données adéquate doit donc être capable d'exprimer cette hiérarchie, et de distinguer ces trois cas, car ils se comportent différemment lors de l'unification :

— une *variable* peut être substituée par n'importe quel terme ;

— une *constante* est un symbole fixe sans arguments ;

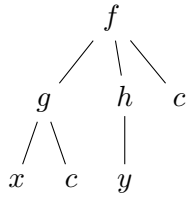
— une *fonction* est un symbole appliqué à un nombre précis de sous-termes, déterminé par son arité.

Notons que, formellement, une constante est une fonction d'arité 0. Nous choisissons néanmoins de les distinguer explicitement, cette séparation étant naturelle et utile en pratique lors de l'unification.

1.2. Intuition

La structure d'un terme est naturellement hiérarchique : une fonction est composée de sous-termes, qui peuvent eux-mêmes être des fonctions, et ainsi de suite. Cette imbrication définit la *profondeur* du terme. Les variables et les constantes, en revanche, sont toujours des *feuilles*, i.e. elles n'ont pas de sous-termes et donc pas de profondeur.

Considérons le terme $t = f(g(x, c), h(y), c)$, où f est une fonction d'arité 3, g une fonction d'arité 2, h une fonction d'arité 1, x, y des variables et c une constante. La structure hiérarchique de t se représente naturellement sous forme d'arbre :



On constate que t est de profondeur 2 : la racine f est au niveau 0, les termes $g(x, c)$ et $h(y)$ ainsi que la feuille c au niveau 1, et enfin x, c et y, des feuilles, au niveau 2. C'est cette structure hiérarchique qui nous amène naturellement à proposer les *arbres* comme structure de données pour représenter un terme.

1.3. Implémentation

La structure hiérarchique d'un terme nous amène donc naturellement à le représenter par un *arbre*. Testons cette idée sur la base B définie comme :

$$B = \{a,\ X,\ Y,\ c,\ f(X, Y),\ g(Y),\ g(a),\ f(a, X)\}$$

Cette représentation capture bien la hiérarchie des termes. Cependant, elle présente deux limites importantes. D'une part, rien ne distingue la variable X de la constante a, ce sont deux feuilles syntaxiquement identiques. D'autre part, rien n'indique qu'un nœud f doit avoir exactement n fils, l'arité de la fonction n'est pas encodée. Or ces deux informations sont essentielles lors de l'unification : une variable peut être substituée, une constante non ; et deux fonctions ne peuvent être unifiées que si elles ont la même arité.

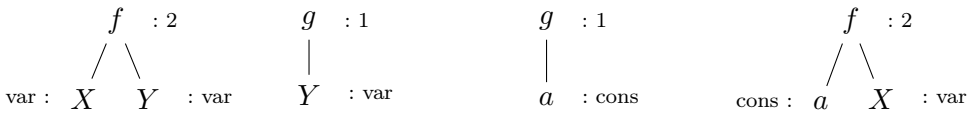
La solution naturelle est d'enrichir l'arbre en associant à chaque nœud une *étiquette* indiquant son type. On définit trois étiquettes possibles :

— var : le nœud est une variable, c'est nécessairement une feuille ;

— cons : le nœud est une constante, c'est nécessairement une feuille ;

— $n \in \mathbb{N}^*$: le nœud est une fonction d'arité n, il a exactement n fils.

Notre base B est maintenant représentée comme ceci :



Cette représentation enrichie permet désormais d'exprimer explicitement toutes les contraintes d'un terme : le *type* de chaque nœud est visible, l'arité des fonctions est encodée dans l'*étiquette*, et la structure hiérarchique est capturée par l'*arbre* lui-même.

Remarque. *Les arbres étant un cas particulier de graphes, cette structure plus générale constitue une piste d'exploration naturelle. En représentant les termes par des graphes avec partage de sous-termes, la détection de cycles lors du parcours pourrait offrir une détection de l'*occur-check *quasi-native. Il s'agit là d'un embryon d'idée que nous n'avons pas eu l'occasion d'explorer dans le cadre de ce rapport.*

2. Structures de données représentant un ensemble de termes

Maintenant que la représentation d'un terme est acquise, nous cherchons à évaluer quelle structure de données est la plus *efficace* pour unifier un terme donné avec un grand nombre d'autres termes. Nous avons ainsi retenu trois structures principales pour nos comparaisons : les *listes*, les *ensembles* et les *dictionnaires*.

2.1. Liste

Principe général.

La liste est la structure séquentielle la plus élémentaire pour stocker notre base de termes. Elle conserve l'ordre d'insertion et autorise la présence de doublons.

Avantages.

- **Coût de construction quasi nul :** L'ajout d'un terme dans la base d'unification se fait en $O(1)$. Cela peut être avantageux dans les cas ou l'on souhaite unifier sans faire de prétraitement, sur une base construite à la volée et peu réutilisée.
- **Parcours brut rapide :** La nature séquentielle de la liste favorise une itération très efficace, le surcoût lié à l'accès en mémoire lors du passage d'un terme à l'autre est quasi inexistant.

Inconvénients.

- **Comparaison exhaustive obligatoire :** La contrepartie de cette simplicité est l'obligation d'une recherche aveugle. Pour unifier un candidat, il est impératif de le confronter un à un à tous les éléments de la liste ($O(N)$ tentatives). Si l'on souhaite filtrer les termes incompatibles pour éviter ce balayage systématique, un prétraitement devient alors obligatoire.
- **Redondance des calculs :** La liste n'empêchant pas les doublons, si un même terme est présent plusieurs fois, l'algorithme d'unification sera exécuté de manière répétée et inutile.

Bilan pour l'unification.

La liste restant la structure séquentielle de référence par excellence, il était naturel de l'envisager en premier. Même en anticipant ses limites de performances sur de grands volumes de données, cette approche naïve reste toutefois intéressante à analyser.

2.2. Ensemble

Principe général.

Un ensemble (*Set*) est une collection non ordonnée qui garantit l'unicité de ses éléments. Dans notre contexte, il s'assure qu'aucun terme strictement identique n'est stocké deux fois dans la base.

Avantages.

- **Élimination native des doublons :** C'est son atout principal. En filtrant les termes redondants dès l'insertion, l'ensemble réduit mécaniquement la taille N de la base. Cela garantit qu'un candidat ne sera jamais unifié deux fois avec le même terme, économisant potentiellement du temps.
- **Vérification d'existence immédiate :** Si l'on cherche une égalité stricte plutôt qu'une unification (par exemple lors de la subsomption), l'ensemble permet de vérifier la présence d'un terme en $O(1)$ moyen, contre $O(N)$ pour une liste.

Inconvénients.

- **Coût d'insertion des termes :** Pour garantir l'unicité en $O(1)$, lors de l'insertion, l'ensemble vérifie si le terme inséré n'est pas déjà présent, ce qui peut prendre un peu de temps.
- **Absence de filtrage pour l'unification :** Tout comme la liste, l'ensemble ne permet pas de sous-sélectionner les candidats à l'unification. Il faut toujours itérer sur l'intégralité de l'ensemble, menant aussi à $O(N)$ tentatives.

Bilan pour l'unification.

L'ensemble améliore l'approche naïve en assainissant la base de données (pas de doublons). Le compromis se situe entre le temps supplémentaire passé lors de la phase d'insertion et le temps gagné lors de la phase de résolution (moins d'unifications inutiles). Toutefois, le problème du parcours exhaustif n'est toujours pas résolu.

2.3. Dictionnaire

Principe général.

Le dictionnaire permet de stocker des paires clé-valeur. Dans le cadre de l'unification, il introduit une première forme d'*indexation de termes* : au lieu de stocker les termes en vrac, on les classe dans le dictionnaire en utilisant une propriété structurelle comme clé (par exemple, le symbole de tête et son arité).

Avantages.

- **Filtrage des candidats :** C'est la première structure qui permet de réduire le nombre de tentatives d'unification. Si l'on cherche à unifier le terme $f(x, y)$, on interroge le dictionnaire avec la clé `'f/2'`. On récupère instantanément uniquement les termes de la base partageant cette même racine.
- **Gain de performance en résolution :** En évitant les tentatives d'unification avec des termes structurellement incompatibles (clash de symboles), on passe d'une complexité de recherche de $O(N)$ à $O(K)$, où K est le nombre de termes partageant la même clé (avec $K \ll N$).

Inconvénients.

- **Complexité et coût mémoire :** Organiser les termes par clés demande davantage d'allocations mémoire (gestion de la table de hachage, listes sous-jacentes pour gérer les collisions ou les groupes de termes).

Bilan pour l'unification.

Le dictionnaire, bien qu'une structure plus complexe, marque la transition entre le stockage simple et l'indexation. En regroupant les termes par « famille » (symbole de tête/arité), il réduit considérablement le nombre d'appels à l'algorithme d'unification. Cette approche profite directement du fonctionnement intrinsèque des termes logiques pour optimiser la recherche, le temps investi lors de la mise en place de cette structure peut être bénéfique pour l'unification.

3. Algorithmes d'unification

3.1. Robinson

Origine et principe..

L'algorithme d'unification de Robinson est un processus algorithmique fondamental en logique mathématique et en informatique théorique. Publié en 1965 par John Alan Robinson, il constitue un pilier du principe de résolution, utilisé pour la démonstration automatique de théorèmes et la programmation logique.

Avant 1965, la démonstration automatique de théorèmes était entravée par l'explosion combinatoire des recherches de preuves. J. A. Robinson a révolutionné le domaine en publiant son article de référence : *A Machine-Oriented Logic Based on the Resolution Principle*. Il y introduit l'unification comme un moyen de sélectionner intelligemment les substitutions nécessaires pour appliquer la règle de résolution, évitant de tester une infinité de combinaisons de manière aveugle.

L'objectif de l'algorithme de Robinson (ainsi que de tous ses successeurs) est de calculer l'*unificateur le plus général* (ou MGU pour *Most General Unifier*) entre deux littéraux, permettant de les rendre identiques via une substitution commune à tous les termes.

Fonctionnement de l'algorithme.

Le déroulement de cet algorithme s'effectue en plusieurs étapes pour unifier deux littéraux :

Algorithm 1 Unification de deux littéraux (Robinson)

```
procedure UnifierLitteraux(L1, L2)
                                        ▷ Étape 1 : Vérification des signes
    if Signe(L1) == Signe(L2) then
        return Échec                    ▷ Pas de résolution possible si même signe
    end if
                                        ▷ Étape 2 : Vérification du nom du prédicat
    if Prédicat(L1) ≠ Prédicat(L2) then
        return Échec                    ▷ Les littéraux n'ont pas le même symbole de prédicat
    end if
                                        ▷ Étape 3 : Vérification de l'arité
    if Arité(L1) ≠ Arité(L2) then
        return Échec                    ▷ Nombre d'arguments différent
    end if
                                        ▷ Étape 4 : Unification itérative des termes (enfants)
    Subst ← ∅                           ▷ Initialisation de la substitution commune
    N ← Arité(L1)
    for i ← 1 à N do
        T1 ← Enfant(L1, i)
        T2 ← Enfant(L2, i)
                                        ▷ Unification en accumulant les substitutions précédentes
        Subst ← UnifierTermes(T1, T2, Subst)
        if Subst == Échec then
            return Échec                ▷ Dès qu'une paire échoue, tout le prédicat échoue
        end if
    end for
    return Subst                        ▷ Retourne la substitution commune (MGU)
end procedure
```

Pour trouver la substitution qui unifie deux termes, on utilise la fonction UnifierTermes, dont le fonctionnement est le suivant :

Algorithm 2 Unification de deux termes (Robinson)

```
procedure UnifierTermes(T_1, T_2, Subst)
                                    ▷ Cas 1 : Les termes ont le même symbole (Nom et Type)
    if Nom(T_1) == Nom(T_2) then
        if EstFonction(T_1) then                    ▷ Cas trivial, pas besoin de substituer
                                    ▷ Unification récursive des arguments (enfants)
            for i ← 1 à Arité(T_1) do
                S_i ← UnifierTermes(Enfant(T_1, i), Enfant(T_2, i))
                if S_i == Échec then
                    return Échec
                end if
                Subst ← Subst ∪ S_i
            end for
            return Subst
        else
            return ∅        ▷ Constantes ou variables identiques : succès, aucune substitution
        end if
    end if
                                    ▷ Cas 2 : Le premier terme est une variable
    if EstVariable(T_1) then
        if OccursCheck(T_1, T_2) then
            return Échec                            ▷ Cycle détecté (T_1 apparaît dans T_2)
        end if
        return {T_1 ↦ T_2}
    end if
                                    ▷ Cas 3 : Le second terme est une variable
    if EstVariable(T_2) then
        if OccursCheck(T_2, T_1) then
            return Échec                            ▷ Cycle détecté (T_2 apparaît dans T_1)
        end if
        return {T_2 ↦ T_1}
    end if
                                    ▷ Cas 4 : Conflit (Clash)
    return Échec                    ▷ Symboles différents ou arités incompatibles
end procedure
```

Remarque. *Cette dernière vérification, appelée occur-check, est une étape essentielle de l'algorithme. Elle empêche d'unifier une variable x avec un terme qui la contient déjà (par exemple, $f(x)$). Sans ce test, l'algorithme entrerait dans une boucle infinie ou produirait des résultats logiquement faux (création d'un terme de taille infinie).*

3.2. Martelli-Montanari

Origine et principe.

L'algorithme Martelli Montanari a été publié en 1982 par Alberto Martelli et Ugo Montanari dans la revue ACM *Transactions on Programming Languages and Systems*. Élément clé dans l'étude de l'unification du premier ordre, sa principale contribution est de modéliser l'unification comme un système de transformation, offrant ainsi un cadre formel qui permet de démontrer la correction de plusieurs algorithmes pratiques. Il surpasse l'algorithme de Robinson en termes d'efficacité. Son objectif est de déterminer, pour un ensemble de termes du premier ordre, s'il existe une substitution qui les rend tous identiques deux à deux, et si tel est le cas, de produire l'unificateur le plus général sous une forme résolue.

Fonctionnement de l'algorithme.

L'algorithme consiste à considérer un système d'équation E et à itérer sur lui jusqu'à atteindre une forme résolue ou détecter un échec. E a pour forme $E = \{s_1 \stackrel{?}{=} t_1, \ldots, s_n \stackrel{?}{=} t_n\}$, où les s_i et t_i sont des termes du premier ordre. Or, dans notre cas, les deux entrées sont des **prédicats** de la forme $P(S_1, \ldots, S_n)$ et $P(t_1, \ldots, t_n)$. Un prétraitement est donc nécessaire afin de pouvoir passer le système d'équation à l'algorithme de Martelli Montanari.

Algorithm 3 Création du système d'équations et unification de littéraux

```
function LIT_LISTE(l1, list_litteraux, touteUnif = True)
    resultat ← ∅
    for l2 ∈ list_litteraux do                ▷ Étape 1 : Comparaison de l1 aux autres littéraux
                                              ▷ Étape 2 : Vérif. symbole, signe et arité
        if l1.predicat = l2.predicat and l1.sign ≠ l2.sign and l1.arity = l2.arity then
            system ← TermSystem()             ▷ Init. du système de termes (équations)
            for (t1, t2) ∈ zip(l1.enfants, l2.enfants) do
                system.add(t1, t2)            ▷ Étape 3 : Ajout des paires d'enfants
            end for
            mm ← MartelliMontanari(system)    ▷ Étape 4 : Résolution via Martelli-Montanari
            try
                unificateur ← mm.solve()
                resultat[l2] ← unificateur
                if not touteUnif then
                    return resultat           ▷ Retour dès le premier succès
                end if
            catch UnificationError
                pass                          ▷ Échec de l'unification, on continue
            end try
        end if
    end for
    return resultat
end function
```

Comment se passe l'unification au sein de Martelli Montanari ?(Étape 4)
On applique les règles suivantes sur le système d'équation jusqu'a épuisement.

— **Delete :** consiste à supprimer une équation réflexive ($t \stackrel{?}{=} t$), c'est-à-dire où les termes de la partie gauche et de la partie droite sont les mêmes.

— **Décompose** $f(s_1, \ldots, s_k) \stackrel{?}{=} f(t_1, \ldots, t_k)$ **:** Lorsque deux termes partagent le même symbole de fonction f et la même arité, il suffit d'unifier leurs sous-termes. On remplace l'équation initiale $f(s_1, \ldots, s_k) \stackrel{?}{=} f(t_1, \ldots, t_k)$ par les équations $s_1 \stackrel{?}{=} t_1, \ldots, s_k \stackrel{?}{=} t_k$. Enfin on procède à son unification composante par composante.

— **Orient** $t \stackrel{?}{=} x$ **:** Par convention, les équations sont orientées de façon à toujours placer la variable à gauche. Si la partie gauche est un terme non-variable et la partie droite est une variable, on échange les deux membres.

— **Occur Check** $x \stackrel{?}{=} f(x)$ **:** consiste à vérifier si la variable x apparaît dans le terme t (sans en être égale), appliquer la substitution $x \to t$ produirait un terme infini, par exemple $x = f(f(f(\ldots)))$. Une telle solution n'existe pas, l'algorithme échoue puis retourne ECHEC.

— **Clash** $f(s_1, \ldots, s_n) \stackrel{?}{=} g(t_1, \ldots, t_m)$ **:** Si les deux termes ont des symboles de tête différents, aucune substitution ne pourra jamais les rendre égaux. L'équation n'a donc pas de solution, l'algorithme s'arrête et retourne ECHEC. Cette règle couvre également le fait de vouloir unifier une constante avec un terme composé ($a \stackrel{?}{=} f(x)$).

Une fois que toutes les règles de transformation ont été appliquées avec succès, le système d'équations est réduit à une forme résolue, c'est-à-dire un ensemble d'équations de la forme $\{x_1 \stackrel{?}{=} t_1, \ldots, x_k \stackrel{?}{=} t_k\}$, où chaque x_i est une variable distincte n'apparaissant dans aucun des termes t_j.

La substitution est alors construite en lisant directement ces liaisons : chaque variable x_i est associée au terme t_i auquel elle a été liée au cours du processus. Cette substitution $\sigma = \{x_1 \mapsto t_1, \ldots, x_k \mapsto t_k\}$ constitue le MGU du système initial.

4. Discrimination Tree

D'après Term Indexing - *R. Sekar, I.V. Ramakrishnan et A. Voronkov,* An Algorithm for the Retrieval of Unifiers from Discrimination Trees - *H. De Nivelle et* On the Evaluation of Indexing Techniques for Theorem Proving - *R. Nieuwenhuis, T. Hillenbrand, A. Riazanov et A. Voronkov*

Le *discrimination tree*, ou *arbre de discrimination* en français (bien qu'aucun ouvrage francophone n'y fasse explicitement référence) est une véritable chimère algorithmique : une *structure de données arborescente* conçue spécifiquement pour l'indexation de termes (ou pour notre cas de prédicats) et un *algorithme* efficace d'unification. C'est en raison de sa nature particulière que nous décidons de la présenter au lecteur dans cette section qui lui est complètement dédiée.

Le *discrimination tree* nous a été présenté par nos encadrants, sans qui cette structure serait vraisemblablement restée dans l'ombre de notre étude. En effet, cette structure ne figurait pas dans notre périmètre initial : c'est précisément grâce à leurs conseils avisés et à la documentation qu'ils nous ont fournie que nous avons pu en saisir le potentiel et décider de l'intégrer pleinement à notre

sujet. Ce choix s'est imposé naturellement au regard de ses propriétés théoriques : le *discrimination tree* se révèle en effet plus puissant que les approches algorithmiques classiques pour l'unification, justifiant ainsi amplement son inclusion. Nous les remercions à nouveau chaleureusement pour cette piste décisive.

La structure arborescente.

Un *discrimination tree* est construit en insérant les termes de la base sous forme aplatie (*flatten*) : chaque terme est d'abord linéarisé en une *p-string* par un parcours préfixe, transformant ainsi sa structure hiérarchique en une séquence de symboles. Ces *p-strings* sont ensuite insérées dans un *arbre*, de sorte que les préfixes communs à plusieurs termes soient *factorisés* au maximum en un unique chemin partagé, évitant toute redondance et constituant ainsi l'index.

Il existe plusieurs variantes du *discrimination tree*, se distinguant notamment par la façon dont les variables sont traitées lors de la construction de l'index. Nous traiterons ici de la variante dite à *variables normalisées* : lors de la linéarisation d'un terme, chaque variable est remplacée par un symbole générique de la forme $*_i$, où i reflète l'ordre d'apparition de la variable dans le parcours préfixe. Ainsi, un terme $f(X, Y)$ devient $f - *_1 - *_2$ et un terme $f(X, X)$ devient $f - *_1 - *_1$. Ce choix facilite le filtrage, i.e. la sélection rapide des termes potentiellement unifiables avec un terme requête, lors de l'unification : la structure de l'index ne dépend plus des noms des variables, mais uniquement de leur *pattern* d'occurrence.

Lorsqu'on travaille avec des prédicats, on ajoute en tête de la *p-string* un symbole indiquant le *signe* du littéral : $+$ pour un prédicat positif $P(\ldots)$ et $\neg$ pour sa négation $\neg P(\ldots)$. Ainsi $P(f(X, Y))$ devient $+-P-f-*_1-*_2$ et $\neg P(f(X, Y))$ devient $\neg-P-f-*_1-*_2$. Cette distinction est essentielle pour la résolution : on ne cherche à unifier un littéral qu'avec des littéraux de signe opposé.

Exemple.

Prenons la base B définie comme suit :

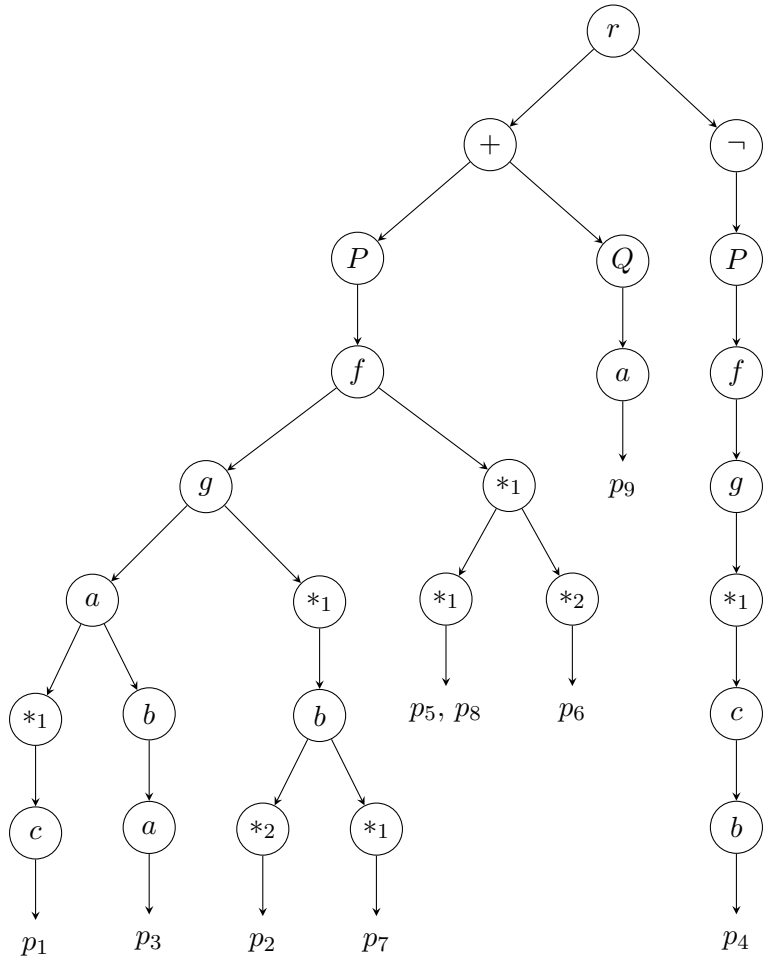
$$B = \{ \underbrace{P(f(g(a, X), c))}_{p_1}, \quad \underbrace{P(f(g(X, b), Y))}_{p_2}, \quad \underbrace{P(f(g(a, b), a))}_{p_3}, \quad \underbrace{\neg P(f(g(X, c), b))}_{p_4},$$

$$\underbrace{P(f(X, X))}_{p_5}, \quad \underbrace{P(f(X, Y))}_{p_6}, \quad \underbrace{P(f(g(X, b), X))}_{p_7}, \quad \underbrace{P(f(Y, Y))}_{p_8}, \quad \underbrace{Q(a)}_{p_9} \}$$

On transforme ces prédicats en *p-strings normalisées* :

$$\begin{array}{lcl}
p_1 & \longrightarrow & + - P - f - g - a - *_1 - c \\
p_2 & \longrightarrow & + - P - f - g - *_1 - b - *_2 \\
p_3 & \longrightarrow & + - P - f - g - a - b - a \\
p_4 & \longrightarrow & \neg - P - f - g - *_1 - c - b \\
p_5 & \longrightarrow & + - P - f - *_1 - *_1 \\
p_6 & \longrightarrow & + - P - f - *_1 - *_2 \\
p_7 & \longrightarrow & + - P - f - g - *_1 - b - *_1 \\
p_8 & \longrightarrow & + - P - f - *_1 - *_1 \\
p_9 & \longrightarrow & + - Q - a
\end{array}$$

L'exemple précédent nous donne donc le *discrimination tree* suivant :



Avec cet exemple, on perçoit déjà l'utilité de la normalisation puisque les *p-strings normalisées* de p_5 et p_8 sont les mêmes. Ainsi l'arbre final ne produit qu'un chemin pour les eux prédicats.

> **Remarque.** *Cet exemple est tiré de* Term Indexing *- R. Sekar, I.V. Ramakrishnan et A. Voronkov. Il a par ailleurs été adapté à notre version* normalisée *du discrimination tree.*

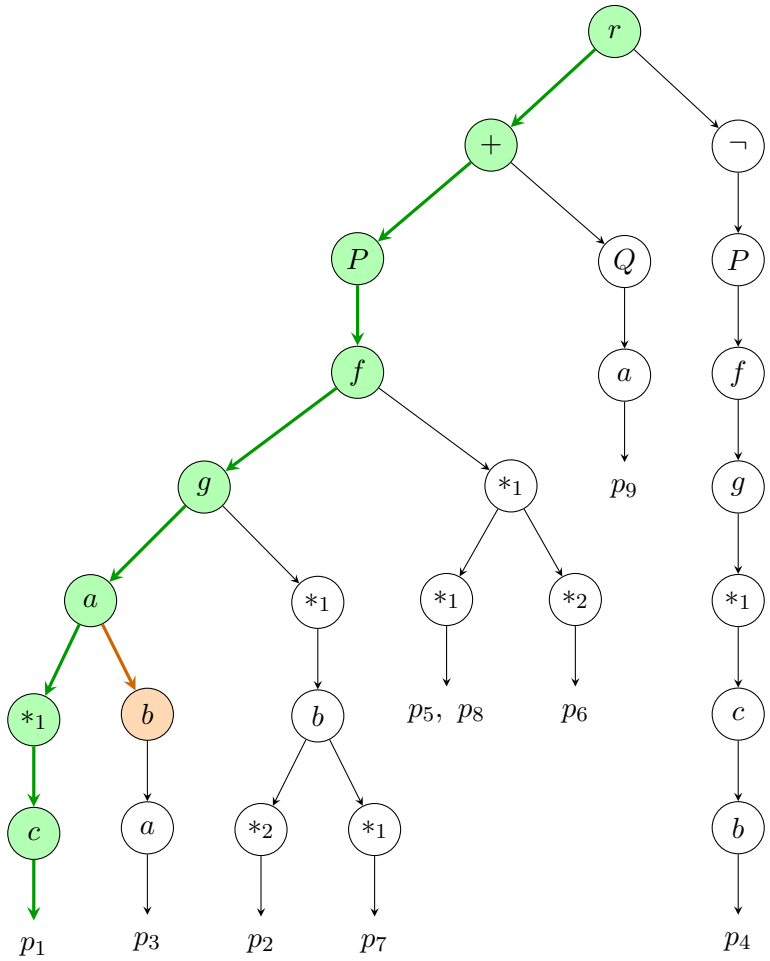
L'algorithme d'unification.

L'unification par *discrimination tree* se déroule en deux phases distinctes. La première phase, dite de *filtrage*, consiste à parcourir l'index à partir d'un prédicat requête q, lui-même linéarisé en *p-string* par parcours préfixe. Les variables de q sont normalisées différemment de celles de l'index : on utilise la notation $?_i$ (contre $*_i$ pour les prédicats indexés), où i reflète l'ordre d'apparition dans le parcours préfixe. Cette distinction est essentielle : lors du filtrage, une variable $?_i$ du terme requête est compatible avec *tout* nœud de l'arbre, symbole concret ou variable $*_j$, tandis qu'un symbole concret de q ne peut correspondre qu'à un nœud identique. À l'issue de ce parcours, on obtient un ensemble de *candidats*, i.e. les prédicats de la base dont la *p-string* est compatible avec celle de q. Il est a noté que pour notre version adaptée à la résolution de prédicats, si la *p-string* de q commence par $\neg$ (respectivement $+$) on filtrera les *p-strings* en cherchant le signe opposé : $+$ (respectivement $\neg$).

La seconde phase, dite d'*unification*, applique alors un algorithme d'unification classique (Robinson) entre q et chacun des candidats retenus, confirmant ou infirmant l'existence d'un unificateur. L'intérêt de cette approche réside dans le fait que le filtrage élimine rapidement et à moindre coût les termes non-unifiables, réduisant considérablement le nombre d'unifications effectives à réaliser.

Considérons le prédicat requête $q = \neg P(f(g(a, Z), c))$, linéarisé et normalisé en $\neg - P - f - g - a - ?_1 - c$.

Phase 1 — Filtrage. On descend dans le *discrimination tree* en suivant la *p-string* $\neg - P - f - g - a - ?_1 - c$:



- $\neg$: on suit la branche $+$, symbole inverse ;
- P : on suit la branche P, symbole identique ;
- f : on suit la branche f, symbole identique ;
- g : on suit la branche g, symbole identique ;
- a : on suit la branche a, symbole identique ;
- $?_1$: variable du requête, compatible avec tout nœud, on suit les branches $*_1$ (✓) et b (✓) ;
- c : depuis $*_1$, on suit c ✓ ; depuis b, aucune branche $c \Rightarrow$ éliminé.

Le filtrage retient uniquement $p_1 = P(f(g(a, X), c))$ comme candidat.

Phase 2 — Unification. On applique l'algorithme d'unification entre $q = \neg P(f(g(a, Z), c))$ et $p_1 = P(f(g(a, X), c))$:

$$\sigma = \{Z \mapsto X\} \quad \text{(ou symétriquement } \{X \mapsto Z\}\text{)}$$

L'unification réussit avec la substitution σ. Les prédicats $p_2, \ldots, p_9$ n'ont jamais été considérés, illustrant l'efficacité du filtrage.

III. Résultats

1. Protocole de benchmarks

2. Résultats